



Agents de coding ■ Dev / DevOps

Formation Claude Code

De l'installation aux cas d'usage Dev / DevOps

Installation · commandes · Skills & agents · hooks · MCP · plugins · 11 cas d'usage

Les briques de Claude Code

1. Installation
2. Commandes de base
3. Commandes → Skills (+ sous-agents & multi-agents)
4. Hooks & garde-fous
5. MCP — outils & données externes
6. Plugins & marketplaces

Mise en pratique

- 7. **11 cas d'usage** Dev / DevOps (UC1–UC11)
- Capstone — fil rouge end-to-end

Logique

On découvre chaque **brique**, puis on l'assemble dans des **cas d'usage** réels.

À l'issue de la formation, le participant sait...

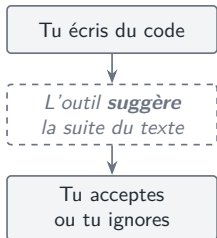
- installer, configurer et **sécuriser** Claude Code sur son poste ;
- piloter une session en maîtrisant les **commandes** et les **permissions** ;
- industrialiser ses usages via **commandes, Skills, hooks et MCP** ;
- orchestrer des **workflows multi-agents** ;
- empaqueter et distribuer un **plugin d'équipe** ;
- traiter des **cas d'usage Dev / DevOps** de bout en bout.

Fil rouge

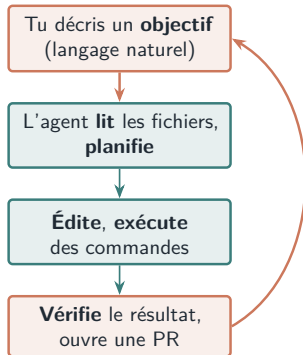
Claude Code n'est pas un assistant de complétion : c'est un agent qui **agit** (lit, édite, exécute, ouvre des PR) sous le contrôle d'un **système de permissions**.

Agentique vs complétion — le changement de paradigme

Complétion classique



Claude Code (agentique)



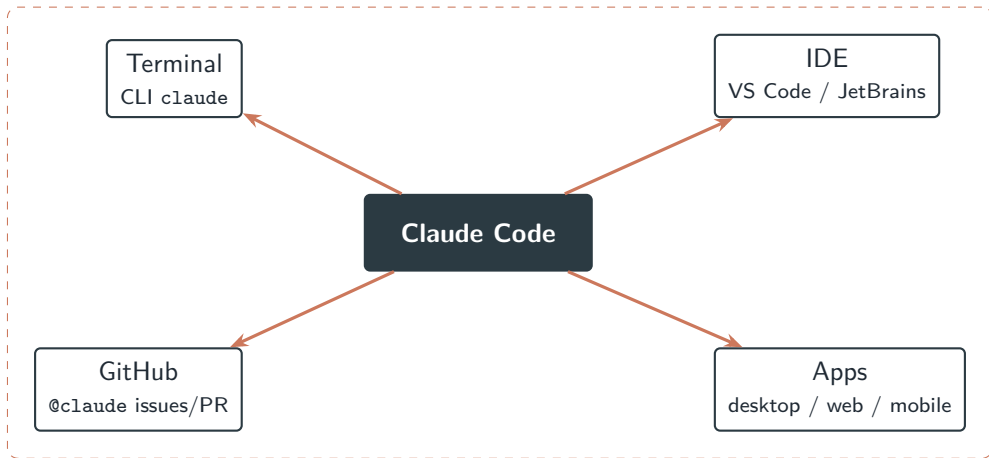
La complétion produit du *texte*. L'agent atteint un *but* — sous contrôle de permissions.

Formation Claude Code

Installation



Où s'exécute Claude Code



Même moteur agentique, plusieurs points d'entrée. Le **contexte projet** ([CLAUDE.md](#), settings) le suit.

Poste participant

- OS : macOS 13+, Linux (Ubuntu 20.04+) ou Windows 10+
- 4 Go de RAM, **Git** installé
- Node.js $\geq$ 18 (uniquement si install via npm)

Compte

- Abonnement **Pro / Max / Team / Enterprise**
- ou **Console** / clé API
- ou fournisseur : Bedrock, Vertex, Foundry

Source d'autorité (note de fraîcheur)

Claude Code évolue vite. Vérifiez toujours : `claude -help`, la commande `/help` dans l'outil, et code.claude.com/docs.

Installation — méthodes recommandées

Plateforme	Commande recommandée
macOS / Linux / WSL	<code>curl -fsSL https://claude.ai/install.sh bash</code>
macOS (Homebrew)	<code>brew install --cask claude-code</code>
Windows (PowerShell)	<code>irm https://claude.ai/install.ps1 iex</code>
Windows (WinGet)	<code>winget install Anthropic.ClaudeCode</code>
npm (Node.js $\geq$ 18)	<code>npm install -g @anthropic-ai/claude-code</code>

```
# Puis, dans le dossier du projet :  
cd mon-projet  
claude                # 1er lancement -> connexion via le navigateur  
# Decouverte : /help (commandes) et /status (compte, version)
```


Vérifier, se connecter & mettre à jour

```
claude --version      # version installée
claude doctor         # diagnostic install + config + connectivité
claude update         # appliquer tout de suite la dernière version
```

- Connexion / déconnexion en session : `/login`, `/logout`.
- Les installeurs natifs se **mettent à jour automatiquement** en arrière-plan.
- Ne **jamais** faire `sudo npm install -g` (risques de permissions/sécurité).

Atelier

Installer, s'authentifier, ouvrir le dépôt sandbox, lancer une session et demander : « *explique-moi l'architecture de ce projet* ».

Formation Claude Code

Commandes de base



Une commande, c'est quoi ?

En clair

Une **commande** = un **raccourci** tapé en début de message (`/nom`) qui déclenche une action ou un workflow. C'est **toi** qui la lances — comportement **déterministe**.

Deux familles

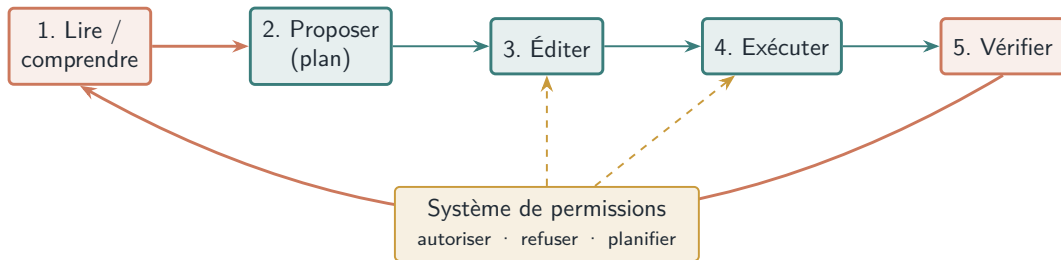
- **Intégrées** — livrées avec l'outil (`/help`, `/init`, `/model...`).
- **Personnalisées** — tes fichiers Markdown dans `.claude/commands/`.

À quoi ça sert

- accélérer une action répétée ;
- standardiser un geste d'équipe ;
- paramétrer via `$ARGUMENTS`.

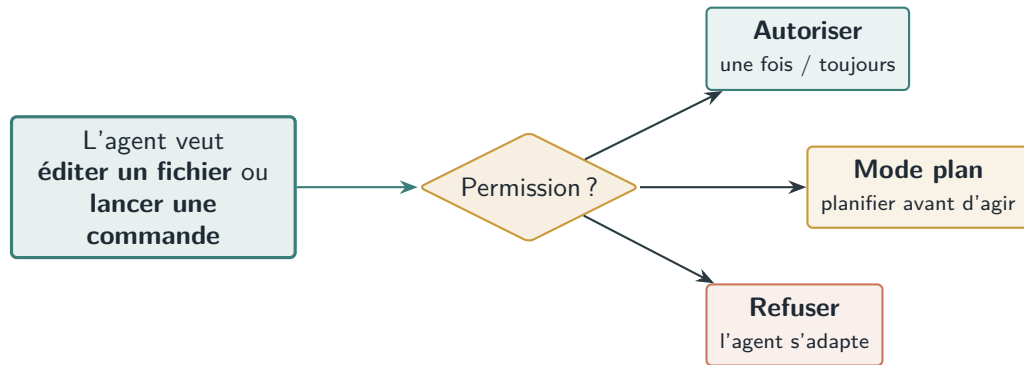
Taper `/` ouvre le menu ; le texte après le nom est passé en argument à la commande.

La boucle de session



Les étapes qui *modifient* (édite) ou *agissent* (exécute) passent par les permissions.

Le modèle de permissions



Règle d'or

Ne jamais « tout autoriser » aveuglément — surtout en contexte bancaire (DORA, traçabilité).

Commandes intégrées les plus utiles

Liste indicative — taper / pour le menu, confirmer via /help.

Repérage & contexte

- /help — aide /status — état
- /init — générer un `CLAUDE.md`
- /memory — éditer la mémoire
- /context — usage du contexte
- /compact — compacter /clear — repartir

Modèle & réglages

- /model — choisir le modèle
- /config — réglages /usage — coût

Sécurité & revue

- /permissions — règles allow/ask/deny
- /plan — mode plan
- /diff — voir les changements
- /review · /code-review ·
/security-review

Extensions & sessions

- /agents · /mcp · /plugin
- /resume · /rewind /exit

Saisies spéciales & modes de lancement

Préfixes (en session)

- @ — référencer un fichier/dossier
- ! — exécuter une commande shell
- # — ajouter une note à la mémoire
- / — ouvrir le menu des commandes

Modes de lancement

- claude — session interactive
- claude "consigne" — démarre avec un prompt
- claude -p "... " — *headless* (script/CI)
- claude -c / -r — continuer / reprendre

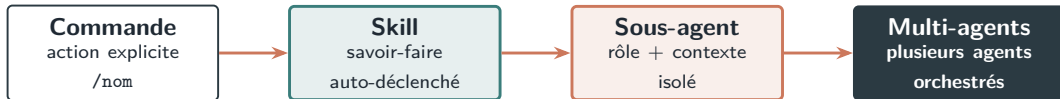
```
@src/Program.cs    explique ce fichier
!dotnet test        # lance les tests, le resultat entre dans la session
# toujours lancer les tests avant un commit  <- ajoute a CLAUDE.md
```

Formation Claude Code

Commandes → Skills, sous-agents & multi-agents

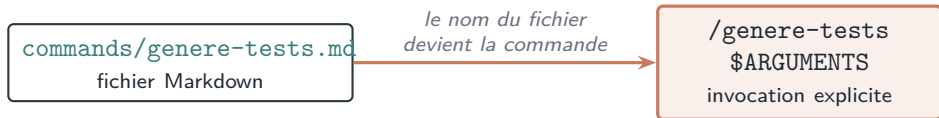


Une échelle de personnalisation



Du raccourci ponctuel à l'orchestration : on monte en puissance *et* en autonomie. Tout reste versionnable dans `.claude/`.

Commandes slash personnalisées



- **Arguments** via `$ARGUMENTS` pour paramétrer la commande.
- Commandes **de projet** (versionnées, partagées) vs **personnelles**.
- Exemples Dev / DevOps : `/gen-tests`, `/containerize`, `/scan-image`.

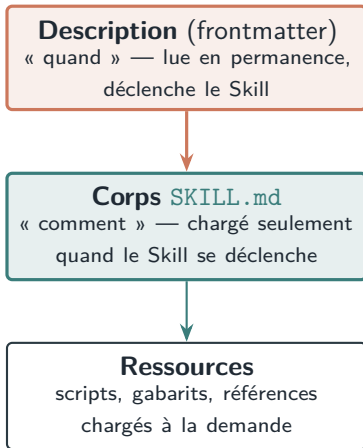
Skill vs commande slash

	Commande slash	Agent Skill
Déclenchement	Explicite (/nom)	Automatique (contexte)
Usage type	Action répétée à la demande	Savoir-faire / conventions
Contrôle	Utilisateur	Modèle (via description)

Exemple DevOps

Skill « conventions conteneurs internes » : Dockerfile multi-stage non-root, nommage & tags d'images, port standard — s'applique dès qu'on touche un [Dockerfile](#) ou un manifeste K8s.

Agent Skill — divulgation progressive



*Économie de contexte :
on ne charge le détail
que si c'est utile.*

Un **Skill** = un dossier **SKILL.md** (+ ressources). Une description précise et « déclenchante » est la clé.

Un **Skill** = un dossier avec un **SKILL.md**. Le **frontmatter** dit *quand* l'activer ; le **corps** dit *comment*.

```
---
name: containerize
description: Genere un Dockerfile multi-stage non-root pour un service
.NET. A utiliser des qu'on veut conteneuriser un microservice.
---
# Procedure
1. Identifier le .csproj du service et ses dependances.
2. Generer un Dockerfile multi-stage (SDK build -> runtime chisele).
3. Executer en non-root (USER), exposer le port 8080.
4. Construire l'image, la scanner (Trivy) et proposer des correctifs.
```

Claude le charge **tout seul** quand la *description* colle au contexte — sans l'invoquer.

En clair

Un **sous-agent** = un **assistant spécialisé** appelé par l'agent principal, avec son **propre contexte**, ses **outils** et ses **permissions**.

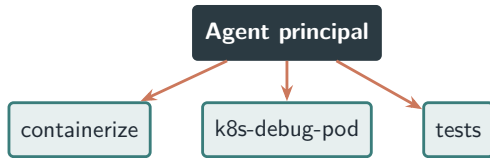
Pourquoi

- **Spécialisation** : un rôle clair (explorer, reviewer, tester).
- **Isolation** : l'« explorateur » ne pollue pas le contexte de l'« architecte ».

Quand l'utiliser

- sous-tâche délimitée ou parallélisable ;
- à arbitrer : **coût et latence**.

Géré via `/agents`.

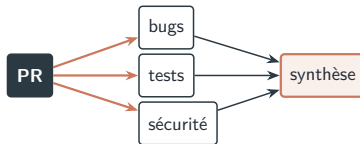


Trois patterns d'orchestration multi-agents

Pipeline
séquentiel



Agents
parallèles



Boucle
itérative



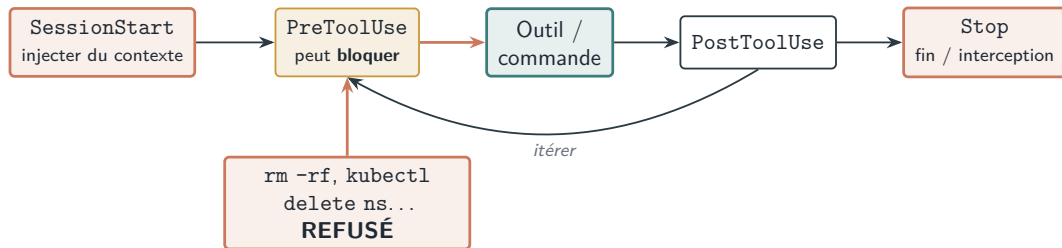
Formation Claude Code

Hooks & garde-fous



En clair

Un **hook** = un **script** lancé **automatiquement** à un moment du cycle de vie. Comme il s'exécute à *tous les coups*, c'est un garde-fou **fiable** : un PreToolUse qui sort en code $\neq 0$ **bloque** l'action.



Les **hooks** (dossier [hooks/](#)) sont des gestionnaires d'événements **déterministes** — un garde-fou qui ne dépend pas du « bon vouloir » du modèle.

Hook — exemple de garde-fou PreToolUse

Un hook PreToolUse peut inspecter une commande et la **bloquer** avant exécution.

```
{
  "hooks": {
    "PreToolUse": [
      { "matcher": "Bash",
        "command": ".claude/hooks/deny-destructive.sh" }
    ]
  }
}
```

deny-destructive.sh : refuse rm -rf, kubectl delete ns, helm uninstall...
-> code de sortie != 0 => l'action est bloquee, l'agent en est informe

Cadre bancaire (DORA / RGPD)

Garde-fous typiques : refuser les actions destructrices, **scanner les secrets** (gitleaks) avant commit, **journaliser** chaque action de l'agent pour l'audit.

Formation Claude Code

MCP — outils & données externes



En clair

MCP (Model Context Protocol) est un **standard** pour brancher Claude à des outils et données externes — le « **USB-C de l'IA** » : une seule prise, beaucoup d'appareils.

Sans MCP

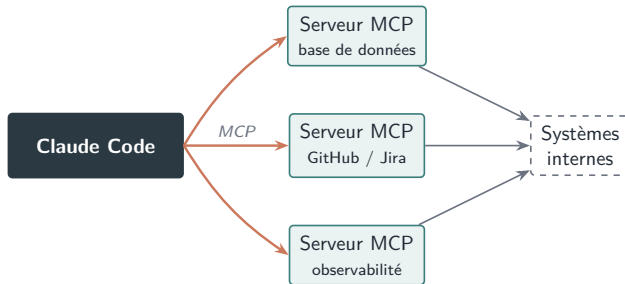
- une intégration sur-mesure par outil ;
- copier-coller manuel des données ;
- fragile, non réutilisable.

Avec MCP

- un **serveur MCP** par système (GitHub, BDD, Jira...);
- Claude (**client**) découvre ses outils tout seul ;
- réutilisable et partageable.

Modèle **client-serveur** : Claude Code est le *client* ; chaque *serveur* MCP expose des **outils** et des **ressources** de façon contrôlée.

Architecture MCP



Model Context Protocol : un protocole **standard** pour un accès *contrôlé* à des outils et données externes.

Configurer un serveur MCP & sécurité

```
# .mcp.json (projet, versionne) -- declare les serveurs MCP
{
  "mcpServers": {
    "github": { "command": "npx", "args": ["-y", "@modelcontextprotocol/server-
      github"] }
  }
}
# Gestion en session : /mcp (lister, connecter, authentifier)
```

Sécurité MCP

Ne connecter que des serveurs **de confiance**. Attention à l'**exfiltration** de données et aux **injections** via contenu externe. Revoir les permissions et tracer les accès.

Formation Claude Code

Plugins & marketplaces



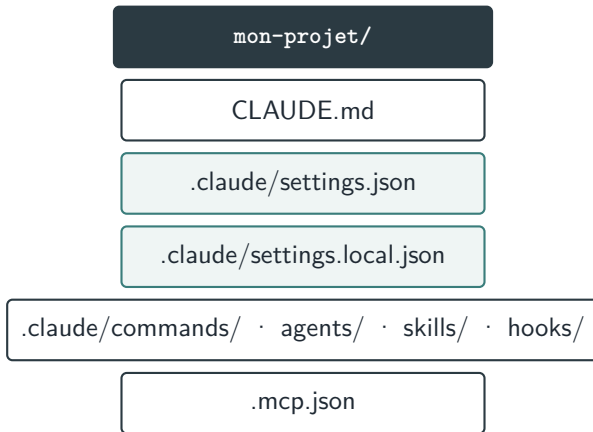
Plugins — emballer & distribuer

- Un **plugin** regroupe commandes, sous-agents, Skills, hooks et serveurs MCP en une unité **partageable et versionnable**.
- Gestion via /plugin; **marketplaces** officielles et internes.
- Objectif : **standardiser** l'usage sur toute l'équipe.

Cible

Le plugin « **ODDO-DevOps-Toolkit** » :
skills k8s-bootstrap, k8s-debug-pod,
postmortem + hooks de garde + MCP
GitHub.





Versionné (partagé) : `CLAUDE.md`, `settings.json`, `commands/`, `agents/`, `skills/`, `hooks/`. Local : `settings.local.json`.

Formation Claude Code

Cas d'usage Dev / DevOps (UC1–UC11)



Panorama — 11 cas d'usage

1 · Code, collaboration & qualité

UC1 Git & GitHub · UC2 CI/CD · UC3 Tests

2 · Conteneurs & orchestration

UC4 Docker · UC5 Cluster K8s · UC6 Helm

3 · Données & application

UC7 Base de données · UC8 UI / frontend

4 · Performance & fiabilité

UC9 Load test · UC10 Observabilité · UC11 Incident & logs

Chaque cas mobilise les briques vues : commandes, Skills, sous-agents, hooks, MCP — et alimente le capstone. **Bonus** : networking, backup/restore, GitOps, FinOps, documentation.

Notions clés (1/2) — code, conteneurs & orchestration

- **PR (Pull Request)** — demande de fusion d'une branche vers `main`, **relue et approuvée** avant intégration.
- **CI/CD** — *pipeline* qui **build, teste, scanne et déploie** à chaque changement.
- **Tests** — **unitaires** (un bout isolé) et **d'intégration** (vrais endpoints, ex. `WebApplicationFactory`).
- **Docker** — empaquette une app + ses dépendances en **image**, lancée en **conteneur** isolé (scan Trivy).
- **Kubernetes (K8s)** — **orchestrateur** de conteneurs : déploie, redémarre, met à l'échelle, gère le réseau.
- **Cluster** — l'ensemble des machines (**nœuds**) pilotées par K8s (ex. k3d en local).
- **Helm / chart** — le « **gestionnaire de paquets** » de K8s; un **chart** = un déploiement paramétrable.

Notions clés (2/2) — données, performance & fiabilité

- **Base de données** — schéma, **migrations** (EF Core) et **seed** (jeu de données de test anonymisé).
- **Test de charge** — simuler du trafic pour trouver les limites. **k6** : l'outil ; **VUs** : utilisateurs virtuels ; **p95** : latence des 95 % requêtes les plus rapides.
- **Observabilité** — voir l'état du système : **métriques** (Prometheus), **dashboards** (Grafana), requêtes PromQL.
- **Post-mortem / runbook** — compte-rendu d'incident / procédure ; **MTTR** = temps moyen de réparation.

Bloc 1 — Code, collaboration & qualité

UC1 — Git & GitHub

On développe : workflow Git assisté + PR propre (branche, commit conventionnel, description, checklist) + hook anti-secret.

Avec Claude : il rédige commits et descriptions, résume la PR et signale les risques — fini la plomberie Git manuelle.

UC2 — CI/CD

On développe : pipelines GitHub Actions (build .NET, tests, scan Trivy, push GHCR) + déploiement k3d, gate manuel avant prod.

Avec Claude : un pipeline complet et correct en minutes, au lieu d'heures de YAML et d'essais-erreurs.

UC3 — Tests

On développe : tests d'intégration sur les vrais endpoints (WebApplicationFactory, base in-memory) + couverture + cas manquants.

Avec Claude : couvre cas nominaux et limites, repère les manques — la couverture monte sans corvée.

Bloc 2 — Conteneurs & orchestration

UC4 — Docker / images

On développe : Dockerfiles multi-stage non-root (chiselé) + `docker-compose` de dev.

Avec Claude : réduit la taille d'image et corrige les findings Trivy — conteneurs plus légers et plus sûrs.

UC5 — Cluster Kubernetes

On développe : cluster k3d (3 nœuds, registre, ingress) + déploiement (Deployments, Services, probes, HPA).

Avec Claude : génère les manifests et diagnostique un pod en CrashLoop (logs + events) — des heures de YAML et de debug en moins.

UC6 — Helm / packaging

On développe : chart Helm paramétrable (values dev/preprod) à partir des manifests.

Avec Claude : factorise et paramètre le déploiement — un seul chart pour tous les environnements.

UC7 — Création de base de données

On développe : schéma Postgres (produits, catégories, stock) + migrations EF Core + jeu de données de seed anonymisé.

Avec Claude : conçoit le schéma, génère migrations et données de test, et optimise les requêtes (lecture du plan d'exécution).

UC8 — UI / frontend à améliorer

On développe : page catalogue modernisée (responsive, états de chargement, a11y) + dashboard de santé des services + maquette.

Avec Claude : propose une maquette et le code — l'IHM progresse même sans expert front dédié.

Le frontend ([ECommerce.Web](#)) et la base servent de terrain de jeu réaliste pour ces deux cas.

Bloc 4 — Performance & fiabilité

UC9 — Load generator (test de charge)

On développe : scénario k6 (paliers jusqu'à 500 VUs, seuil p95 < 300 ms) + rapport + analyse.

Avec Claude : écrit le scénario et lit le rapport pour pointer le goulot — on trouve les limites avant la prod.

UC10 — Observabilité

On développe : Prometheus + Grafana + dashboards + requêtes PromQL + règles d'alerte.

Avec Claude : génère dashboards et alertes (latence, erreurs, saturation) — visibilité en place rapidement.

UC11 — Gestion d'incident & analyse de logs

On développe : runbook + template de post-mortem à partir d'une vraie timeline d'incident.

Avec Claude : clusterise 50k lignes de logs et propose la cause racine — MTTR réduit, post-mortem blameless rédigé.

Bonus : networking (Nginx, TLS), backup & restore, GitOps (ArgoCD/Flux), FinOps, documentation (runbooks, ADR).

Ce qu'on gagne avec Claude Code

Gains concrets

- **Vitesse** : un Dockerfile, un chart Helm, un pipeline CI en **minutes** au lieu d'heures.
- **Moins d'erreurs** : l'agent **lit, teste, itère**; les **hooks** bloquent les gestes dangereux.
- **Montée en compétence** : un profil junior produit (et **comprend**) du Docker/K8s de niveau senior.
- **Diagnostic** : logs analysés, cause racine & post-mortem → **MTTR réduit**.

À l'échelle de l'équipe

- **Standardisation** : conventions capitalisées en Skills / commandes / **plugin**.
- **Documentation** : runbooks, ADR, diagrammes générés **automatiquement**.
- **Conformité** : scans, détection de secrets, **journal d'audit** (DORA/RGPD).
- **Focus** : moins de plomberie répétitive, plus de **valeur métier**.

L'ingénieur reste aux commandes

L'agent **propose et exécute sous permissions**; l'humain **valide, teste et tranche** — surtout sur les actions sensibles (apply, prod, merge).

Capstone — fil rouge end-to-end

Livrer l'`ecommerce-app` de la conteneurisation à la prod observée, **uniquement avec Claude Code** et les briques construites.

1. Conteneuriser (UC4), cluster K8s (UC5), Helm (UC6).
2. Base de données + migrations + seed (UC7).
3. Déployer en CI/CD avec gate (UC2).
4. Tester : unitaires/intégration (UC3) + charge k6 (UC9).
5. Observer : Prometheus/Grafana + alerting (UC10).
6. Simuler un incident → post-mortem (UC11).
7. Embaquer le **plugin ODDO-DevOps-Toolkit** & partager.

Restitution

Chaîne complète + garde-fous (hooks) + 3 cas prioritaires + KPI fiabilité (DORA) (MTTR, MTTD, change failure rate) + plan 30/60/90 jours.

- Documentation : <https://code.claude.com/docs/en/overview>
- Installation & configuration : <https://code.claude.com/docs/en/setup>
- Commandes : <https://code.claude.com/docs/en/commands>
- Skills · Hooks · MCP · Plugins : code.claude.com/docs (sections dédiées)
- Usage des données : <https://code.claude.com/docs/en/data-usage>
- En session : `/help`, `/status`, `claude -help`

Place aux labs

Des briques aux 11 cas d'usage.



Bassem Ben Hamed